

# Titanic Dataset Preprocessing for Model Training

April 9, 2025

## Introduction

This project begins with preprocessing the Titanic dataset to prepare it for supervised machine learning. The goal is to clean the data, handle missing values, encode categorical variables, and scale numerical features so that the dataset becomes compatible with modeling algorithms such as logistic regression.

## Step 1: Import Required Libraries

Code

```
# 1. Import required libraries  
import pandas as pd  
from sklearn.linear_model import LogisticRegression
```

## Step 2: Data Loading and Preprocessing

### Code

```
from sklearn.preprocessing import LabelEncoder, StandardScaler

# Load the dataset
df_final = pd.read_csv('Titanic-Dataset.csv')

# Drop columns that are not useful for modeling
df_final = df_final.drop(columns=['PassengerId', 'Name', 'Ticket', 'Cabin'])

# Handle missing values
df_final['Age'] = df_final['Age'].fillna(df_final['Age'].median())
df_final['Embarked'] = df_final['Embarked'].fillna(df_final['Embarked'].mode()[0])

# Encode 'Sex' using LabelEncoder
le = LabelEncoder()
df_final['Sex'] = le.fit_transform(df_final['Sex'])

# One-hot encode 'Embarked'
df_final = pd.get_dummies(df_final, columns=['Embarked'], drop_first=True)

# Standardize numerical features
scaler = StandardScaler()
df_final[['Age', 'Fare']] = scaler.fit_transform(df_final[['Age', 'Fare']])
```

### Explanation:

This code performs a full preprocessing pipeline to transform the Titanic dataset into a clean, numeric format suitable for machine learning models.

- **Data Loading:** The dataset is read into a pandas DataFrame using `pd.read_csv()`.
- **Column Dropping:** Columns such as `PassengerId`, `Name`, `Ticket`, and `Cabin` are removed. These columns either represent unique identifiers or contain too many missing values (like `Cabin`), and thus do not contribute meaningful information for prediction.
- **Handling Missing Values:**
  - `Age` is filled with its median value, which is ideal for numeric features that may contain outliers.
  - `Embarked` is filled with the most frequent port (mode), which makes sense as only a couple of rows are missing.
- **Encoding Categorical Variables:**
  - `Sex` is encoded using `LabelEncoder`, turning 'male' and 'female' into 1 and 0, respectively.
  - `Embarked` is one-hot encoded. This converts it into binary columns, with one of them dropped (`drop_first=True`) to avoid the dummy variable trap (redundancy).
- **Standardizing Numerical Features:** The `Age` and `Fare` columns are scaled using `StandardScaler`, which transforms them to have a mean of 0 and standard deviation of 1. This scaling is essential when features are on different scales, especially for distance-based models or those sensitive to feature magnitude (like logistic regression, SVMs, etc.).

This preprocessing pipeline ensures that the dataset is:

- Entirely numerical
- Free from missing values
- Appropriately scaled
- Ready for model training and evaluation

By applying best practices in data cleaning and transformation, this stage lays a solid foundation for reliable and interpretable machine learning results.

## Step 3: Splitting the Dataset for Model Training and Testing

### Code

```
# Split Data for Training and Testing
from sklearn.model_selection import train_test_split

X = df_final.drop('Survived', axis=1) # Features
y = df_final['Survived']              # Target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

### Explanation:

This code splits the preprocessed Titanic dataset into **training and testing sets**, which is an essential and foundational step before fitting any machine learning model. It ensures that we can both train the model and independently evaluate its performance on unseen data.

- **X** contains the **features** — all input variables used to predict survival (i.e., all columns except **Survived**).
- **y** is the **target** variable — the actual outcome, where 1 indicates survival and 0 indicates non-survival.

The function `train_test_split()` from `sklearn.model_selection` performs the partitioning:

- **X\_train, y\_train**: 80% of the dataset is allocated for model training. This subset is used by the algorithm to learn patterns and build the model.
- **X\_test, y\_test**: 20% of the dataset is set aside for testing. This part is not seen by the model during training and is used to evaluate how well the model generalizes to new, unseen data.

The parameter `random_state=42` is set to ensure **reproducibility**. Every time this code is executed, the data will be split in the same way, which is especially important when comparing results across different models or experiments.

Splitting the data in this way prevents **data leakage**, where the model might learn from data it should not have access to during training. It also enables a fair and consistent way to measure model performance using metrics such as accuracy, precision, recall, or ROC-AUC.

This step prepares the Titanic dataset for supervised learning models like **logistic regression**, **decision trees**, **random forests**, or more advanced techniques such as ensemble methods or neural networks.

It is crucial to maintain this structure (train/test split) throughout the machine learning pipeline to ensure the integrity and generalizability of the results.

## Step 4: Lasso Logistic Regression and Feature Coefficients

### Code

```
df = pd.read_csv('Titanic-Dataset.csv')

lasso_model = LogisticRegression(penalty='l1', solver='liblinear')
# 'liblinear' solver supports L1 regularization

lasso_model.fit(X_train, y_train)

lasso_coeffs = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': lasso_model.coef_[0]
})

lasso_coeffs.sort_values(by='Coefficient', ascending=False)
```

### Output: Coefficients from L1-Regularized Logistic Regression

| Feature    | Coefficient |
|------------|-------------|
| Fare       | 0.137554    |
| Embarked_Q | 0.000000    |
| Parch      | -0.095114   |
| SibSp      | -0.290685   |
| Embarked_S | -0.300348   |
| Age        | -0.373984   |
| Pclass     | -0.901768   |
| Sex        | -2.607504   |

### Explanation:

This step applies **logistic regression with L1 regularization (also known as Lasso)** to the preprocessed Titanic dataset. Regularization is used to reduce model complexity, prevent overfitting, and in the case of L1, also perform **automatic feature selection** by forcing less important feature coefficients to zero.

- **penalty='l1'** applies Lasso regularization, which tends to shrink less relevant feature coefficients to exactly 0, effectively removing them from the model.
- **solver='liblinear'** is necessary because it is one of the solvers that supports L1 regularization in scikit-learn.
- After training the model using **.fit()**, we extract the learned coefficients associated with each feature using **model.coef\_**.
- A pandas DataFrame is created to pair each feature with its corresponding coefficient and sort them by magnitude for interpretability.

From the output, we can observe:

- **Sex** has the most negative impact on survival with a coefficient of -2.61, indicating that being male (Sex = 1) strongly decreases the chance of survival.
- **Fare** is the only feature with a positive coefficient, suggesting that higher ticket prices (which often correlate with higher social class or better cabin access) increase survival likelihood.

- `Embarked_Q` has a coefficient of 0, meaning it was likely not useful and effectively removed from the model by L1 regularization.

This method not only helps in training a robust model but also aids in **understanding which features are truly influential** for predicting passenger survival on the Titanic.

## Step 5: Feature Selection Using Recursive Feature Elimination (RFE)

### Code

```
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression

# Initialize RFE with Logistic Regression as the estimator
rfe = RFE(estimator=LogisticRegression(solver='liblinear'), n_features_to_select=5)

# Fit RFE on the training data
rfe.fit(X_train, y_train)

# Extract the selected features
selected_features = X.columns[rfe.support_]

# Display the result
print("Selected by RFE:", list(selected_features))
```

### Explanation:

This step demonstrates how to use **Recursive Feature Elimination (RFE)** for selecting the most informative features in the Titanic dataset. RFE is a wrapper-based feature selection technique that recursively removes the least important features based on a model's learned coefficients.

- **Model Setup:** `LogisticRegression` with `solver='liblinear'` is used as the base estimator. Logistic regression is appropriate here due to its interpretability and efficiency with binary classification problems.
- **RFE Initialization:** RFE is configured to select the top 5 features from the dataset. It works by:
  - Training the model on the current set of features
  - Ranking features based on their importance (magnitude of coefficients)
  - Eliminating the least important feature
  - Repeating this process recursively until the desired number of features remains
- **Fitting the Model:** The method `rfe.fit(X_train, y_train)` fits RFE using the training data, allowing it to evaluate the contribution of each feature to model accuracy.
- **Selecting Features:** After fitting, `rfe.support_` returns a boolean mask indicating which features were selected. These features are extracted using `X.columns` and printed.

Using RFE helps simplify the model by removing redundant or less informative features. It is especially useful when dealing with datasets where many variables may be correlated or not significantly contributing

to prediction performance. By retaining only the top-performing features, models can often generalize better, run faster, and be more interpretable.

This step complements previous feature selection techniques like Lasso regression by offering a different, iterative approach that relies directly on the model's performance during each round of elimination.

## Step 6: Logistic Regression Model Training and Evaluation

### Code

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report

# Select final features
selected_features = ['Pclass', 'Sex', 'Age', 'SibSp', 'Embarked_S']
X_selected = df_final[selected_features]
y = df_final['Survived']

# Split into training and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X_selected, y, test_size=0.2, random_state=42)

# Train the logistic regression model
model = LogisticRegression(solver='liblinear')
model.fit(X_train, y_train)

# Make predictions on the test set
y_pred = model.predict(X_test)

# Evaluate using confusion matrix and classification report
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

### Output:

#### Confusion Matrix:

```
[[89 16]
 [21 53]]
```

#### Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.81      | 0.85   | 0.83     | 105     |
| 1            | 0.77      | 0.72   | 0.74     | 74      |
| accuracy     |           |        | 0.79     | 179     |
| macro avg    | 0.79      | 0.78   | 0.78     | 179     |
| weighted avg | 0.79      | 0.79   | 0.79     | 179     |

### Explanation:

In this step, a logistic regression model is trained on a selected subset of features from the Titanic dataset to predict passenger survival. The features chosen — `Pclass`, `Sex`, `Age`, `SibSp`, and `Embarked_S` — were either identified as important through feature selection or are known to be highly predictive.

- The dataset is split into an 80% training set and a 20% test set using `train_test_split`, ensuring evaluation on unseen data.
- The logistic regression model is trained using `solver='liblinear'`, which is efficient for small to medium datasets and supports L1 and L2 regularization.
- Predictions on the test set are generated using `model.predict()`.
- Model performance is evaluated using a **confusion matrix** and a **classification report**.

#### **Confusion Matrix Interpretation:**

- True Negatives (TN): 89 passengers correctly predicted as not survived.
- False Positives (FP): 16 passengers incorrectly predicted as survived.
- False Negatives (FN): 21 passengers incorrectly predicted as not survived.
- True Positives (TP): 53 passengers correctly predicted as survived.

#### **Classification Metrics:**

- **Accuracy (79%):** Proportion of total correct predictions out of all predictions.
- **Precision:** Among predicted positives, how many are actually positive.
- **Recall:** Among actual positives, how many were correctly predicted.
- **F1-score:** Harmonic mean of precision and recall; balances the two in case of class imbalance.

The classification report shows balanced performance across both classes, with slightly higher precision and recall for class 0 (non-survivors). The model demonstrates reasonable effectiveness, particularly given the simplicity of the features used. These metrics are crucial for understanding how the model performs and whether improvements (e.g., using more features or tuning hyperparameters) are needed.

## Step 7: Multiclass Logistic Regression with Custom Survival Classes

### Code

```
# Predict survival probabilities
y_probs = model.predict_proba(X_selected)[: , 1]

# Create 3-class survival labels based on custom thresholds
import numpy as np
survival_class = np.select(
    [y_probs < 0.4, (y_probs >= 0.4) & (y_probs < 0.7), y_probs >= 0.7],
    [0, 1, 2]
)

# Check the distribution of the classes
from collections import Counter
print("Class-Distribution:", Counter(survival_class))

# Train/test split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X_selected, survival_class, test_size=0.2, random_state=42
)

# Train multinomial logistic regression model
from sklearn.linear_model import LogisticRegression
multi_model = LogisticRegression(multi_class='multinomial', solver='lbfgs')
multi_model.fit(X_train, y_train)

# Train One-vs-Rest logistic regression model
ovr_model = LogisticRegression(multi_class='ovr', solver='liblinear')
ovr_model.fit(X_train, y_train)

# Evaluate both models
from sklearn.metrics import classification_report
print("Multinomial-Logistic-Regression-Model-Evaluation:\n",
      classification_report(y_test, multi_model.predict(X_test)))

print("One-vs-Rest-Logistic-Regression-Model-Evaluation:\n",
      classification_report(y_test, ovr_model.predict(X_test)))
```

### Output:

#### Class Distribution:

Counter({0: 528, 1: 185, 2: 178})

#### Multinomial Logistic Regression Model Evaluation:

|   | precision | recall | f1-score | support |
|---|-----------|--------|----------|---------|
| 0 | 0.98      | 0.99   | 0.98     | 97      |
| 1 | 0.91      | 0.93   | 0.92     | 42      |
| 2 | 0.97      | 0.93   | 0.95     | 40      |



|              |      |      |      |     |
|--------------|------|------|------|-----|
| accuracy     |      |      | 0.96 | 179 |
| macro avg    | 0.95 | 0.95 | 0.95 | 179 |
| weighted avg | 0.96 | 0.96 | 0.96 | 179 |

#### One-vs-Rest Logistic Regression Model Evaluation:

|   | precision | recall | f1-score | support |
|---|-----------|--------|----------|---------|
| 0 | 0.87      | 1.00   | 0.93     | 97      |
| 1 | 0.88      | 0.55   | 0.68     | 42      |
| 2 | 0.90      | 0.93   | 0.91     | 40      |

  

|              |      |      |      |     |
|--------------|------|------|------|-----|
| accuracy     |      |      | 0.88 | 179 |
| macro avg    | 0.88 | 0.82 | 0.84 | 179 |
| weighted avg | 0.88 | 0.88 | 0.87 | 179 |

#### Explanation:

In this step, we explore **multiclass classification** using logistic regression by creating **custom 3-class survival labels** based on predicted survival probabilities:

- `y_probs` stores the predicted survival probabilities (probability of class 1) from the previously trained binary logistic regression model.
- Using `np.select()`, we classify each prediction into one of three classes:
  - Class 0: Low chance of survival (probability  $\leq 0.4$ )
  - Class 1: Medium chance (0.4 < probability  $\leq 0.7$ )
  - Class 2: High chance (probability > 0.7)
- This allows us to shift from a binary classification problem to a **custom-defined ordinal problem** with three tiers of risk.

We then split the dataset again and train two types of logistic regression models:

#### 1. Multinomial Logistic Regression (softmax):

- Implemented via `multi_class='multinomial'` with solver `'lbfgs'`.
- Directly handles multiclass predictions through a softmax function.
- Achieves an impressive 96% accuracy with strong performance across all three classes.

#### 2. One-vs-Rest (OvR) Logistic Regression:

- Uses a binary logistic regression for each class vs. all others.
- Implemented via `multi_class='ovr'` and solver `'liblinear'`.
- While still solid, the performance is slightly lower, especially for the middle class (Class 1) where recall drops to 0.55.

#### Evaluation Metrics:

- **Precision** indicates how many predictions of a class were correct.
- **Recall** tells us how many actual instances of a class were correctly identified.
- **F1-score** balances precision and recall, giving a holistic view of model performance.
- **Macro average** treats all classes equally, while **weighted average** considers class imbalance.

The results show that **\*\*multinomial logistic regression\*\*** is better suited for this task than the One-vs-Rest approach. This is likely due to its ability to model class probabilities jointly, rather than through multiple independent binary classifiers.

**Note:** Warnings related to **multi\_class** deprecation indicate that newer versions of scikit-learn will use **multinomial** as the default. It's best practice to remove this parameter in future updates for compatibility.

This step demonstrates how probability thresholds and multiclass classification can be used together to create more interpretable, granular survival categories—useful in real-world scenarios where "high", "medium", and "low" risk assessments are more actionable than binary outcomes.

## Step 8: Interpreting Logistic Regression Coefficients and Odds Ratios

### Code

```
import numpy as np
import pandas as pd

# Get coefficients for binary logistic regression or first class in multinomial
coefficients = model.coef_[0]

# Create a DataFrame to show the Feature, Coefficient, and Odds Ratio
coef_summary = pd.DataFrame({
    'Feature': X_selected.columns,
    'Coefficient': coefficients,
    'Odds-Ratio': np.exp(coefficients)
})

# Sort by impact (absolute coefficient)
coef_summary = coef_summary.loc[
    coef_summary['Coefficient'].abs().sort_values(ascending=False).index
]

# Display the result
print(coef_summary)
```

**Output:**

| Feature    | Coefficient | Odds Ratio |
|------------|-------------|------------|
| Sex        | -2.438310   | 0.087308   |
| Pclass     | -0.896237   | 0.408103   |
| Age        | -0.351338   | 0.703746   |
| Embarked_S | -0.318381   | 0.727326   |
| SibSp      | -0.273327   | 0.760844   |

### Explanation:

This section interprets the **\*\*coefficients\*\*** from a trained logistic regression model and converts them into **\*\*odds ratios\*\***, which are often more intuitive for understanding the magnitude and direction of feature influence.

- The **coefficient** represents the change in the log-odds of the target (i.e., survival) for a one-unit increase in the feature, holding all other features constant.
- The **odds ratio**, calculated as  $e^{\text{coefficient}}$ , shows the multiplicative effect on the odds of survival. An odds ratio:
  - Greater than 1 implies a positive influence on survival (increases odds)
  - Less than 1 implies a negative influence (decreases odds)
  - Equal to 1 means no effect

#### Interpretation of Key Results:

- **Sex:** With an odds ratio of 0.087, being male ( $\text{Sex} = 1$ ) significantly decreases the odds of survival. This is the most impactful feature.
- **Pclass:** Higher passenger classes (encoded as lower numbers) correspond to better survival chances. A one-unit increase in **Pclass** reduces the odds of survival by about 60%.
- **Age**, **Embarked\_S**, and **SibSp** also negatively impact survival odds, though to a lesser extent.

This summary provides both a statistical and practical understanding of which variables most affect survival prediction. By examining both the coefficients and the odds ratios, we bridge the gap between model training and real-world interpretability — a crucial component in applied data science.